

A walkthrough of `pollinatorR`

Pollinator-dependence analytics for exclusion experiments, end to end

Max Moldovan

2026-06-05

Contents

1	What this walkthrough is	1
2	The worked dataset	2
3	Act I – the analyst	2
3.1	Declare the study once	2
3.2	Estimate the contribution	3
3.3	Attribute it to the community	3
4	Act II – the reviewer	4
4.1	Indices, but with an interval	4
4.2	The coupling trap	4
4.3	Was the yield gained or merely moved	4
4.4	Integrity before inference	4
5	Reproduce it yourself	5
6	Notes and limitations	5
7	Session information	5

1 What this walkthrough is

`pollinatorR` turns a pollinator-exclusion experiment – paired open and caged plots – into a defensible estimate of what insect pollinators contribute to yield, with the uncertainty carried through rather than discarded. This document runs the whole package on one worked dataset, from declaring the study to the integrity checks a reviewer would demand. A reader should be able to follow it in ten minutes and reproduce it in thirty.

Two voices run through it. The *analyst* wants a number: how much does pollinator access add, and which insects drive it. The *reviewer* distrusts that number and looks for the ways a pollination study usually goes wrong – a ratio index regressed on a yield axis it shares a term with, an effect that is really an unrecorded site difference, a yield component that was traded away rather than gained.

The data here are simulated, not the field trial: every value below comes from the `set.seed(1)` block in the next section, so nothing in this walkthrough touches real or restricted records.

2 The worked dataset

A modest open-versus-cage design: ten sites, sixteen plants per site, two insect exposures (honeybee and wild-insect counts), and two yield components (pod number and seeds per pod). The data-generating process is deliberate. Open access lifts pod set, and it lifts pods more where honeybees are abundant – a treatment-by-exposure interaction the analyst should recover and the reviewer should be able to trust.

```
ns <- 10L
per <- 16L
Site <- rep(sprintf("s%02d", seq_len(ns)), each = per)
trt <- rep(rep(c("Cage", "Open"), each = per / 2L), ns)
honeybee <- rep(rpois(ns, 8), each = per) + rpois(ns * per, 1)
wild <- rep(rpois(ns, 4), each = per) + rpois(ns * per, 1)
site_eff <- rnorm(ns, 0, 0.3)[as.integer(factor(Site))]
open <- trt == "Open"
pods <- rpois(ns * per, exp(1.8 + 0.35 * open + 0.015 * honeybee * open + site_eff))
seeds_per_pod <- rpois(ns * per, exp(2.6 + 0.12 * open + site_eff))
canola <- data.frame(Site, trt, honeybee, wild, pods, seeds_per_pod)
head(canola)
#>   Site trt honeybee wild pods seeds_per_pod
#> 1 s01 Cage      6    5    7         34
#> 2 s01 Cage      6    5   10         31
#> 3 s01 Cage      7    7   15         17
#> 4 s01 Cage      7    6   13         35
#> 5 s01 Cage      8    7   11         32
#> 6 s01 Cage      7    5   12         25
```

3 Act I – the analyst

3.1 Declare the study once

Every later call reads from one `pollin_spec()` object, so the study definition lives in exactly one place. The `latent_unit` is the unit that carries a random treatment slope: the site-to-site spread in the contribution becomes a parameter rather than noise.

```
spec <- pollin_spec(
  data = canola,
  outcomes = c("pods", "seeds_per_pod"),
  treatment = "trt",
  control = "Cage",
  random = "Site",
  exposures = c("honeybee", "wild"),
  latent_unit = "Site"
)
spec
#> <pollin_spec>
#> observations : 160
#> outcomes      : pods, seeds_per_pod
#> treatment      : trt (control = Cage ; 2 levels)
#> random         : Site [slope on: Site ]
#> exposures      : honeybee, wild
```

3.2 Estimate the contribution

`pollin_contribution()` fits a `glmmTMB` mixed model for each outcome, choosing a count, proportion, or continuous family from the data, and reports the open-minus-exclusion contrast through `marginalEffects`. The estimate is a contribution on the response scale, with a confidence interval and `slope_sd` – the standard deviation of the random treatment slope, which says how much the contribution itself varies between sites.

```
contribution <- pollin_contribution(spec)
contribution
#> <pollin_contribution> (open - exclusion, model-based ATE)
#>      outcome term      contrast estimate std.error      p.value      conf.low
#>      pods   trt Open - Cage 4.859859 0.7663021 2.268573e-10 3.357934097
#> seeds_per_pod trt Open - Cage 1.517424 0.7705741 4.892877e-02 0.007126137
#> conf.high family slope_sd      p.fdr
#> 6.361783 nbinom2 0.05736834 4.537145e-10
#> 3.027721 nbinom2 0.02551147 4.892877e-02
```

With two outcomes the analyst is already testing more than one hypothesis, so the p-values are brought under false-discovery-rate control before anything is called significant.

```
contribution$table$fdr <- pollin_fdr(contribution$table$p.value)
contribution$table[, c("outcome", "estimate", "p.value", "fdr")]
#>      outcome estimate      p.value      fdr
#> 1      pods 4.859859 2.268573e-10 4.537145e-10
#> 2 seeds_per_pod 1.517424 4.892877e-02 4.892877e-02
```

3.3 Attribute it to the community

The next question – which insects drive the contribution – is where naive analyses fail, because a site with many honeybees may differ in a dozen unrecorded ways. `pollin_moderation()` answers it with a within-between (Mundlak) decomposition: each exposure is split into its site mean and the deviation within a site, and only the within-site treatment-by-exposure term is read as the pollinator effect.

```
moderation <- pollin_moderation(spec, outcome = "pods")
moderation
#> <pollin_moderation> outcome: pods (treatment x exposure)
#>      term      estimate std.error      p.value
#> trtOpen:honeybee_within 0.14181372 0.05357523 0.008120894
#> trtOpen:honeybee_between 0.12417284 0.06038076 0.039734644
#> trtOpen:wild_within 0.02543762 0.05574981 0.648187467
#> trtOpen:wild_between 0.07133184 0.06402668 0.265238157
#>      component      p.fdr
#> within (robust) 0.01624179
#> between (vulnerable)      NA
#> within (robust) 0.64818747
#> between (vulnerable)      NA
```

The split is available on its own through `pollin_within_between()`. The `between` component is the site mean; the `within` component is the part identified inside a site, robust to any site-level confounder.

```
wb <- pollin_within_between(canola$honeybee, group = canola$Site)
str(wb)
#> List of 2
#> $ within : num [1:160] -1.125 -1.125 -0.125 -0.125 0.875 ...
#> $ between: num [1:160] 7.12 7.12 7.12 7.12 7.12 ...
```

4 Act II – the reviewer

4.1 Indices, but with an interval

The reviewer accepts the dependence index as a summary, but not as a bare ratio. `pollin_rii()` returns the Relative Interaction Index with a bootstrap interval, and `pollin_index()` adds the log response ratio with a closed-form interval. Here the index is built from per-site open and exclusion pod means.

```
open_mean <- tapply(canola$pods[open], canola$Site[open], mean)
cage_mean <- tapply(canola$pods[!open], canola$Site[!open], mean)
pollin_rii(open_mean, cage_mean, n_boot = 500, seed = 1)
#> RII = 0.240 [95% boot CI: 0.143, 0.330] (n_open=10, n_cage=10)
pollin_index(open_mean, cage_mean, n_boot = 500, seed = 1)
#>
#> index estimate conf.low conf.high
#> 1 RII 0.2398524 0.1425111 0.3302758
#> 2 lnRR 0.4892350 0.2662971 0.7121729
#> 3 dependence_ratio 0.3869048 NA NA
#>
#> note
#> 1 bootstrap CI
#> 2 closed-form CI
#> 3 point; do not regress on open yield (see pollin_guard_spurious)
```

4.2 The coupling trap

The reviewer's sharpest objection: a paper that regresses a dependence index on absolute yield is partly measuring an identity, because the open-arm term sits on both axes (Pearson 1897). `pollin_guard_spurious()` permutes the exclusion arm while holding the open arm fixed, so the permutation distribution is exactly the mechanical-coupling null.

```
pollin_guard_spurious(open_mean, cage_mean, n_perm = 200, seed = 1)
#> <pollin_spurious> index ~ open-yield slope guard
#> observed slope : 0.0168
#> coupling null : mean 0.0391, sd 0.0108 (n_perm = 200)
#> permutation p : 0.990 -> slope consistent with mechanical coupling (SPURIOUS risk)
```

A permutation p-value near one means the observed slope is indistinguishable from coupling – it should not be reported as a finding. The `decouple` = argument lets the analyst regress the index on an axis that shares no term (the exclusion arm, or an independent vigour covariate) and recover an honest slope.

4.3 Was the yield gained or merely moved

A contribution to one component can be a true gain or a re-allocation from another. `pollin_compensation()` summarises the trade-off structure among yield components as a correlation matrix, so a negative off-diagonal flags compensation rather than gain.

```
pollin_compensation(canola[, c("pods", "seeds_per_pod")])
#> <pollin_compensation> ( spearman correlations )
#> a b cor compensation
#> seeds_per_pod pods 0.3775392 FALSE
#> 0 negatively-correlated pair(s) -> candidate compensation
```

4.4 Integrity before inference

Finally, the checks that should run before any model is fitted: missingness, the factor levels the design promises, and any identity the analyst asserts about the data.


```

#> [1] pollinatoR_0.0.0.9000 testthat_3.3.2
#>
#> loaded via a namespace (and not attached):
#> [1] TMB_1.9.19          xfun_0.57            devtools_2.4.6
#> [4] remotes_2.5.0        insight_1.4.6        lattice_0.22-7
#> [7] numDeriv_2016.8-1.1 vctrs_0.7.3          tools_4.5.2
#> [10] Rdpack_2.6.6         generics_0.1.4       parallel_4.5.2
#> [13] sandwich_3.1-1       Matrix_1.7-4         data.table_1.18.2.1
#> [16] checkmate_2.3.4      desc_1.4.3           lifecycle_1.0.5
#> [19] compiler_4.5.2       brio_1.1.5           codetools_0.2-20
#> [22] htmltools_0.5.9      usethis_3.2.1        marginaleffects_0.32.0
#> [25] yaml_2.3.12          nloptr_2.2.1         MASS_7.3-65
#> [28] ellipsis_0.3.2       cachem_1.1.0         reformulas_0.4.4
#> [31] sessioninfo_1.2.3    boot_1.3-32          multcomp_1.4-29
#> [34] nlme_3.1-168         parallelly_1.47.0    digest_0.6.39
#> [37] mutnorm_1.3-6        future_1.70.0        purrr_1.2.2
#> [40] listenv_0.10.1       splines_4.5.2        rprojroot_2.1.1
#> [43] fastmap_1.2.0        grid_4.5.2           cli_3.6.6
#> [46] magrittr_2.0.5       survival_3.8-3       pkgbuild_1.4.8
#> [49] TH.data_1.1-5        backports_1.5.1      estimability_1.5.1
#> [52] rmarkdown_2.31       emmeans_2.0.2        globals_0.19.1
#> [55] otel_0.2.0           lme4_2.0-1           zoo_1.8-15
#> [58] memoise_2.0.1        coda_0.19-4.1        evaluate_1.0.5
#> [61] knitr_1.51           glmmTMB_1.1.14       rbibutils_2.4.1
#> [64] mgcv_1.9-3           rlang_1.2.0          Rcpp_1.1.1-1.1
#> [67] xtable_1.8-8         glue_1.8.1           pkgload_1.5.2
#> [70] rstudioapi_0.18.0    minqa_1.2.8          R6_2.6.1
#> [73] fs_2.1.0

```